



تقرير إنجاز مشروع البرمجة المتوازية – سنة رابعة – اختصاص ذكاء اصطناعي

إعداد الطلاب

بشار سليم الضاهر

علي صلاح حمدان

آية لؤي محمد

الياس ميشيل نادر

Chapter 1 – Introduction

1.1 خلفية عن العمل:

يهدف هذا المشروع إلى بناء نظام خلفي متكامل لمنظومة تجارة إلكترونية قادر على معالجة آلاف الطلبات المتزامنة مع ضمان سلامة البيانات واستقرار الأداء تحت الضغط العالي. يُركز المشروع على تطبيق المتطلبات غير الوظيفية العشرة المدروسة في المادة، وإثباتها بأدلة قابلة للقياس.

النتيجة الرئيسية: انتقل النظام من معدل فشل 6.64% إلى 0% تحت 100 مستخدم متزامن، مع تحسين متوسط وقت الاستجابة بنسبة 55% في الاختبار الشامل.

1.2 أهداف المشروع:

يهدف هذا المشروع إلى تحسين المتطلبات غير الوظيفية لمنصة التجارة الإلكترونية مع الحفاظ على الوظائف الأساسية للنظام.

وتتمثل الأهداف الرئيسية للمشروع فيما يلي:

- تقليل زمن الاستجابة للعمليات الأساسية.
- تخفيض عدد الاستعلامات المرسلة إلى قاعدة البيانات.
- تحسين قابلية التوسع وزيادة قدرة النظام على استيعاب أعداد أكبر من المستخدمين المتزامنين.
- منع حدوث حالات التداخل بين الطلبات المتزامنة أثناء عمليات الشراء.
- ضمان اتساق البيانات والحفاظ على سلامة المخزون.
- توفير طبقة مراقبة لقياس مؤشرات الأداء المختلفة.
- تقييم فعالية التحسينات من خلال اختبارات تحميل واقعية.

1.3 مكونات النظام الرئيسية:

يتكون النظام من طبقات متتالية:

- Client (k6) يُرسل طلبات HTTP محاكاةً لمستخدمين حقيقيين
- Nginx Reverse Proxy يوزع الطلبات على 3 Laravel instances (8001/8002/8003)
- Laravel Instances كل instance يعمل بـ 10 PHP workers
- Redis يخدم Cache + Queue + Distributed Locks
- MySQL قاعدة البيانات الرئيسية مع Pessimistic Locking
- Queue Workers ينفذون الـ Jobs بشكل غير متزامن

1.4 تطبيق AOP:

طُبِّق مفهوم AOP عبر PerformanceMonitorMiddleware بطريقة تفصل قياس الأداء عن الـ Business Logic تماماً:

- `handle() = Before Advice`: يُسجّل وقت البداية والذاكرة، ويُفعّل DB listener لعدد الاستعلامات
 - `terminate() = After Advice`: يحسب `duration_ms + memory_kb + query_count` ويكتبها في AOP log
 - `channelMap`: يوجّه كل نوع اختبار إلى ملف log مستقل حسب X-Test-Type header
- هذا يعني أن أي endpoint يمر عبر `performance.monitor middleware` يُراقب تلقائياً دون تعديل كوده الداخلي.

```
// Before Advice
public function handle(Request $request, Closure $next)
{
    $request->attributes->set('perf_start_time', microtime(true));
    DB::listen(fn() => app()->instance('perf.query_count', $count + 1));
    return $next($request);
}

// After Advice
public function terminate($request, $response)
{
    $duration = round((microtime(true) - $startTime) * 1000, 2);
    Log::channel($channels['aop'])->info('AOP_TERMINATE_MONITOR', [
        'duration_ms' => $duration, 'memory_kb' => $memory,
        'query_count' => $queryCount, 'status_code' => $response->getStatusCode()
    ]);
}
```

Chapter 2 – System Analysis

2.1 المتطلبات الوظيفية:

يمثل هذا القسم المتطلبات الوظيفية الأساسية للنظام، والتي توضح الخدمات الأساسية التي يقدمها نظام التجارة الإلكترونية للمستخدمين. تم تصميم النظام ليكون قادراً على دعم العمليات التجارية الأساسية بشكل كامل وفعال.

يشمل النظام الوظائف التالية:

- تسجيل المستخدمين (User Registration) حيث يمكن للمستخدم إنشاء حساب جديد داخل النظام .
- تسجيل الدخول (Login) للتحقق من هوية المستخدم وإدارة الجلسات .
- البحث عن المنتجات (Product Search) وتمكين المستخدم من الوصول السريع إلى المنتجات المطلوبة .
- عرض المنتجات (Product Viewing) مع تفاصيل كل منتج .
- إضافة المنتجات إلى السلة (Add to Cart) وإدارة محتوياتها .
- تنفيذ عملية الطلب (Place Order) وتحويل السلة إلى طلب فعلي .
- عرض سجل الطلبات (Order History) لتمكين المستخدم من متابعة عمليات الشراء السابقة.

بالإضافة لمتطلبات أخرى لكن تم ذكر أعلاه المتطلبات الأهم.

2.2 المتطلبات غير الوظيفية:

#NFR1 – Concurrent Access & Data Integrity

عند وصول 80 مستخدم في نفس الثانية لشراء نفس المنتج بدون حماية، يقرأ كل مستخدم stock=10 ثم يكتب 9، فتنتهي كميات المخزون بشكل خاطئ.

الدليل من ال Log قبل التحسين:

```
USER 9 READ stock=10
USER 2 READ stock=10
USER 7 READ stock=10

USER 9 WRITE new_stock=9
USER 2 WRITE new_stock=9
USER 7 WRITE new_stock=9
```

الحل المطبق:

- lockForUpdate(): يُقفل صف المنتج في MySQL أثناء العملية
- orderBy('id'): يضمن ترتيب القفل لمنع Deadlock (كل المستخدمين ينفلون بنفس الترتيب)

```
$products = Product::whereIn('id', $productIds)
->orderBy('id') // Synchronization Point: Deadlock Prevention
->lockForUpdate() // Synchronization Point: Pessimistic Lock
->get()->keyBy('id');
```

الاختبار:

```
USER 59 ACQUIRED DB LOCK | READ stock=10 → WRITE 9 → DECREMENTED
USER 6 ACQUIRED DB LOCK | READ stock=9 → WRITE 8 → DECREMENTED
USER 2 ACQUIRED DB LOCK | READ stock=8 → WRITE 7 → DECREMENTED
...
USER 53 ACQUIRED DB LOCK | READ stock=0 → FAILED insufficient stock
USER 58 ACQUIRED DB LOCK | READ stock=0 → FAILED insufficient stock
```

Order_Race_Condition.js: 80 VU × 10 ثوانٍ، كل مستخدم

يُرسَل طلبات متواصلة.

النتيجة: قبل stock = ينقص بشكل غير منضبط | بعد = كل طلب يقرأ stock محدث.

#NFR2 – Resource Management & Capacity Control

التطبيق:

- RateLimiter: 5 محاولات → 429 Too Many Attempts login/ip/phone
- حد السلة 50: منتج مختلف كحد أقصى لكل مستخدم
- حد الكمية 100: وحدة للمنتج الواحد في السلة
- Global Rate Limit: 1000 طلب/دقيقة لـ Place Order على مستوى النظام
- PHP Workers: 10 per instance × 3 instances = 30 worker متزامن

```
$globalExecuted = RateLimiter::attempt(
    'global-system-orders', 1000, fn() => {}, 60 // Capacity Control
);
```

#NFR3 – Asynchronous Queues

ال Jobs المطبقة:

- SendLoginNotification: إرسال إشعار أمني عند تسجيل الدخول
- SendOrderConfirmationJob: تأكيد الطلب للمستخدم
- GenerateInvoiceJob: إنشاء الفاتورة
- ProcessUserImage: معالجة صورة المستخدم عند التسجيل

الاختبار:

قبل التحسين — Login duration = 5,273ms (QUEUE_CONNECTION=sync): ال Job يُنفَّذ داخل الطلب.

بعد التحسين (dispatch()) — Login duration = 166ms (QUEUE_CONNECTION=redis): يرجع فوراً.

```
DB::afterCommit(function () use ($order, $userId) {
    // Synchronization Point: Jobs only after successful commit
    SendOrderConfirmationJob::dispatch($order);
    GenerateInvoiceJob::dispatch($order);
});
```

#NFR4 – Batch Processing

التطبيق:

GenerateDailySalesReportJob يعالج طلبات اليوم السابق على دفعات لمن memory overflow :

```
Order::whereDate('created_at', $date)
    ->with('items')
    ->chunk(500, function ($orders) use (&$stats) {
        $stats['chunks_processed']++;
        // الذاكرة تحرير ثم طلب 500 معالجة
    });
```

```
> \App\Models\DailySalesReport::latest()->first();
= App\Models\DailySalesReport {#8544
  id: 2,
  report_date: "2026-06-03",
  total_orders: 18,
  approved_orders: 10,
  rejected_orders: 5,
  pending_orders: 3,
  total_revenue: "6757.00",
  total_items_sold: 38,
  chunks_processed: 1,
  created_at: "2026-06-03 13:43:52",
  updated_at: "2026-06-03 13:43:52",
}
```

- الجدولة: كل يوم الساعة 02:00 صباحاً تلقائياً عبر Laravel Scheduler
- withoutOverlapping(): يمنع تنفيذ الـ Job مرتين في نفس الوقت (Synchronization Point)

الدليل من Tinker

#NFR5 – Load Distribution

الإعداد:

- 3 Laravel instances: ports 8001, 8002, 8003
- 10 PHP workers per instance = 30 worker
- Nginx – least_conn (اتصالاً الأقل) + keepalive 32

least_conn لأن طلباتنا متفاوتة الوزن Place Order: تأخذ 110ms بينما Search Cache Hit تأخذ 71ms .
مع round_robin سيرفر واحد قد يتراكم عنده طلبات ثقيلة.
least_conn يوجّه الطلب دائماً للسيرفر الأقل انشغالاً.

```
upstream laravel_app {
    least_conn;          # Synchronization Point: Load Balancing Strategy
    keepalive 32;        # Connection reuse
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
    server 127.0.0.1:8003;
}
```

#NFR6 – Distributed Caching

جدول مفاتيح الـ Cache:

Cache Key	المحتوى	المدة	الملف
products:v{ver}:page:{page}	+ المنتجات قائمة صورها	ساعة	ProductService::index()
product:detail:{id}	واحد منتج تفاصيل	ساعة 24	ProductService::optimizedShow()
cart:user:{userId}:page:{page}	المستخدم سلة	دقائق 10	CartService::show()
cart_count:{userId}	السلة في items عدد	دقائق 5	OrderService
user_orders_{userId}_page_{page}	المستخدم طلبات	دقيقة	OrderService::getUserOrders()

search_fulltext:{md5}	البحث نتائج	دقيقة	SearchController
user_auth_data_{phone}	للمستخدم بيانات login	دقائق 10	AuthService::optimizedLogin()
stores:page:{page}	المتاجر قائمة	ساعة	StoreService::index()
store:detail:{id}	متجر تفاصيل	ساعة 24	StoreService::show()
admin:sales_reports:last_{n}_days	المبيعات تقارير	دقيقة 30	SalesReportService

الدليل الرقمي من AOP:

Search Cache Miss: query_count=3, duration=282ms → Search Cache Hit: query_count=0, duration=71ms

Show Product Miss: query_count=2 → Show Product Hit: query_count=0, duration=62ms

```

{"test_type": "search", "time": "2026-06-17 12:56:47", "duration_ms": 155.87, "memory_kb": 5719.76, "query_count": 3, "status_code": 200, "timestamp": "2026-06-17 12:56:47"}
{"test_type": "search", "time": "2026-06-17 12:56:47", "duration_ms": 80.4, "memory_kb": 4134.02, "query_count": 0, "status_code": 200, "timestamp": "2026-06-17 12:56:47"}
{"test_type": "search", "time": "2026-06-17 12:56:49", "duration_ms": 102.26, "memory_kb": 5737.8, "query_count": 3, "status_code": 200, "timestamp": "2026-06-17 12:56:49"}
{"test_type": "search", "time": "2026-06-17 12:56:49", "duration_ms": 69.73, "memory_kb": 4134.0, "query_count": 0, "status_code": 200, "timestamp": "2026-06-17 12:56:49"}
{"test_type": "search", "time": "2026-06-17 12:56:49", "duration_ms": 70.45, "memory_kb": 4134.02, "query_count": 0, "status_code": 200, "timestamp": "2026-06-17 12:56:49"}
{"test_type": "search", "time": "2026-06-17 12:26:51", "duration_ms": 4587.12, "memory_kb": 5738.02, "query_count": 3, "status_code": 200, "timestamp": "2026-06-17 12:26:51"}
{"test_type": "search", "time": "2026-06-17 12:26:51", "duration_ms": 4587.23, "memory_kb": 5768.98, "query_count": 3, "status_code": 200, "timestamp": "2026-06-17 12:26:51"}
{"test_type": "search", "time": "2026-06-17 12:26:51", "duration_ms": 4587.21, "memory_kb": 5738.02, "query_count": 3, "status_code": 200, "timestamp": "2026-06-17 12:26:51"}

```

#NFR7 – Concurrency Control

الأقفال نوعان:

النوع الأول: DB Lock (Pessimistic)

```

Product::whereIn('id', $productIds)
    ->orderBy('id')           // Deadlock Prevention
    ->lockForUpdate()         // SELECT FOR UPDATE
    ->get();

```

اخترنا Pessimistic لأن نسبة التضارب مرتفعة جداً تحت الضغط (80-150 VU يتنافسون على نفس المنتج).
Optimistic يحتاج retry loop عند التضارب مما يُعقد الكود ويُبطئ التجربة.

النوع الثاني: Distributed Lock (Redis)

```

$lock = Cache::lock('place_order:{userId}', 10); // TTL=10s
if (!$lock->get()) return ['error' => 'Processing...', 'status' => 429];
try { /* العملية */ } finally { $lock->release(); }

```

:Thundering Herd Protection

عند Cache Miss على منتج مشهور، ال Distributed Lock يمنع 100 استعمال DB متزامن:

```
Cache::lock('lock:product:{id}', 10)->block(5, function () {
    if (Cache::has($cacheKey)) return; // Already populated
    $product = Product::with('image')->find($id);
    Cache::put($cacheKey, $product->toArray(), now()->addHours(24));
});
```

#NFR8 – ACID Integrity

التطبيق:

كل عملية Place Order تغلف 6 خطوات في Transaction واحدة:

- 1. قراءة السلة (Cart read)
 - 2. قفل المنتجات (lockForUpdate)
 - 3. إنشاء الطلب (Order::create)
 - 4. إدراج ال Items على دفعة (OrderItem::insert — Batch)
 - 5. تقليل المخزون (decrement)
 - 6. حذف السلة (Cart::delete)
- إذا فشلت أي خطوة Rollback تلقائي لكل ما سبق.

```
$order = DB::transaction(function () use ($userId) {
    // Synchronization Point: ACID Transaction
    $cartItems = Cart::where(...)->get();
    $products = Product::lockForUpdate()->get();
    $order = Order::create([...]);
    OrderItem::insert($orderItems); // Batch Insert
    $product->decrement('quantity');
    Cart::where('user_id', $userId)->delete();
    DB::afterCommit(fn() => SendOrderConfirmationJob::dispatch($order));
    return $order;
});
```

#NFR9 – Stress Testing

Combined_100_Users.js

محاكاة واقعية لـ 100 مستخدم متزامن بثلاثة سيناريوهات:

- 30 Browsers: تسجيل دخول + بحث + عرض منتج
 - 50 Shoppers: تسجيل دخول + بحث + عرض + إضافة للسلة
 - 20 Buyers: كل العمليات + تقديم الطلب
- هذا التوزيع يُحاكي الواقع 70-80% من المستخدمين يتصفحون فقط.

المقياس	التحسين قبل	التحسين بعد
الطلبات إجمالي	542	1,176 (+117%)
الفاشلة الطلبات	36 (6.64%)	0 (0.00%)
الاستجابة وقت متوسط	26,219 ms	11,823 ms (-55%)
استجابة وقت أقصى	60,001 ms (Timeout)	14,767 ms
الاستجابة وقت p95	59,999 ms	14,102 ms (-77%)
Checks ال نجاح معدل	95.52%	100.00%
السيناريو	100 VU: 30+50+20	100 VU: 30+50+20

#NFR10 – Benchmarking & Bottleneck Analysis

لاكتشاف الاختناق اعتمدنا:

- البيانات قاعدة في الاختناق → مرتفع query_count + مرتفع duration_ms
- duration_ms + query_count منخفض → الاختناق في الكود في (job sync أو I/O)
- memory_kb متزايد → memory leak محتمل
- status_code=429 متكرر → Resource exhaustion

Endpoint	duration قبل (ms)	duration بعد (ms)	query_count قبل	query_count بعد
Login	5,273	166	4	2
Search (Cache Miss)	282	147	3	3
Search (Cache Hit)	108	71	3	0
Show Product (Miss)	100	117	2	2
Show Product (Hit)	100	62	2	0
Add to Cart	51	71	3	4
Place Order	248	110	8	10

الاختناق الأول: Login Sync Jobs

SendLoginNotification تعمل داخل الطلب

(QUEUE_CONNECTION=sync) → 5,273ms per request.

الحل :

QUEUE_CONNECTION=redis → dispatch() فوراً يرجع (97%-) 166ms.

الاختناق الثاني: Search without index

LIKE '%Hot%' يُجري Full Table Scan → query_count=3, duration=282ms.

الحل :

FULLTEXT Index + Cache::remember → Cache Hit: query_count=0, duration=71ms (-75%).

:Synchronization Points 2.3

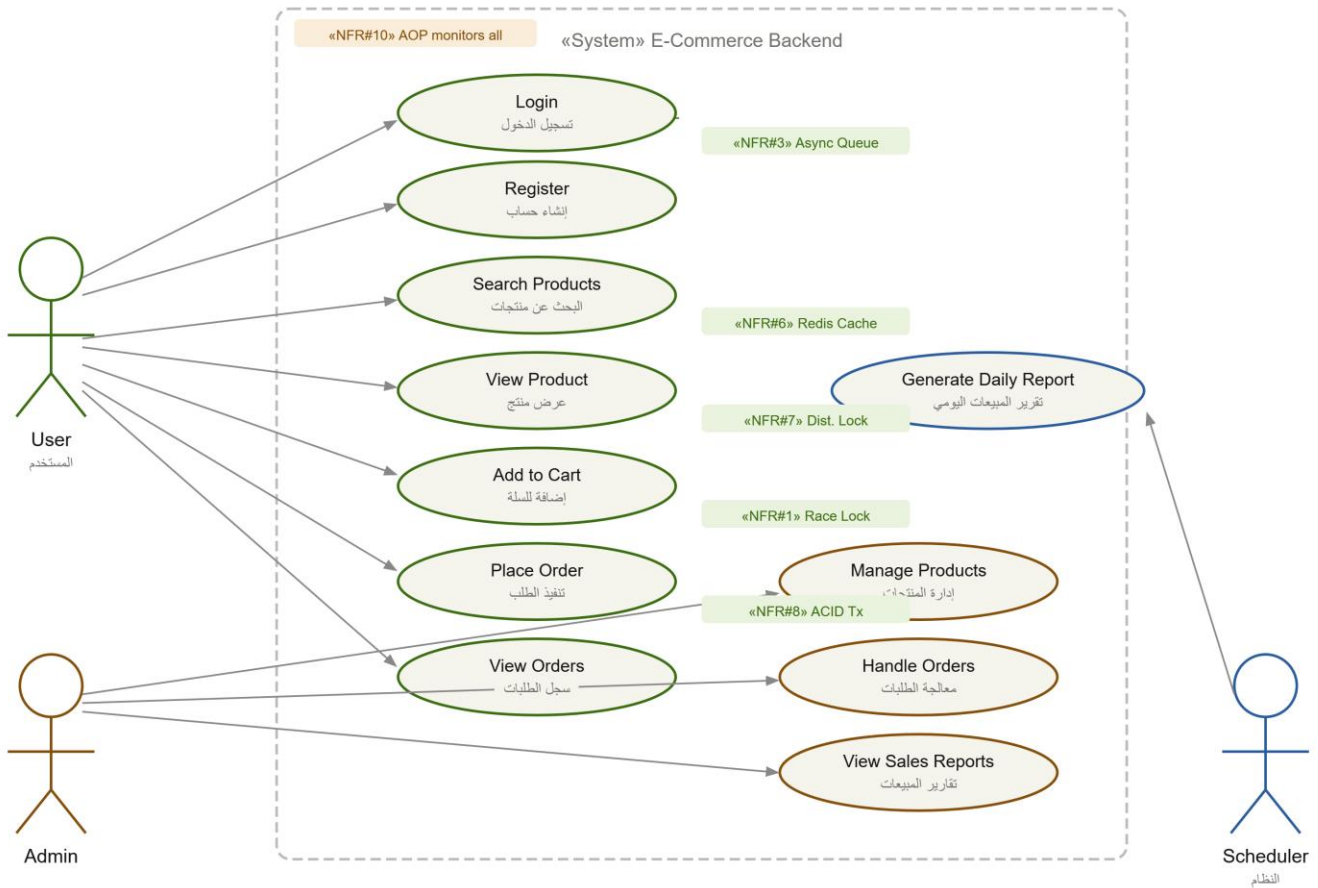
الهدف	المزامنة نوع	الدالة	الملف
ACID - Atomicity ضمان	DB::transaction()	placeOrderOptimized()	OrderService.php
Distributed Lock - منع Duplicate	Cache::lock('place_order:{id}')	placeOrderOptimized()	OrderService.php
Pessimistic Lock - منع Deadlock	lockForUpdate() + orderBy('id')	placeOrderOptimized()	OrderService.php
نجاح بعد Async Jobs Transaction	DB::afterCommit()	placeOrderOptimized()	OrderService.php
Pessimistic Lock على Product + Cart	lockForUpdate() x2	optimizedAdd()	CartService.php
Distributed Lock - منع Thundering Herd	Cache::lock('lock:product:{id}')	optimizedShow()	ProductService.php
Pessimistic Lock الطلبات لمعالجة	lockForUpdate() + orderBy('id')	handleOrder()	Admin/OrderService.php
تعديل بدون الأداء قياس Business Logic	AOP Before/After Advice	handle() + terminate()	PerformanceMonitorMiddleware
Batch Job تنفيذ منع مرتين	withoutOverlapping()	schedule()	Kernel.php

2.4 مخطط حالات الاستخدام – Use Case Diagram

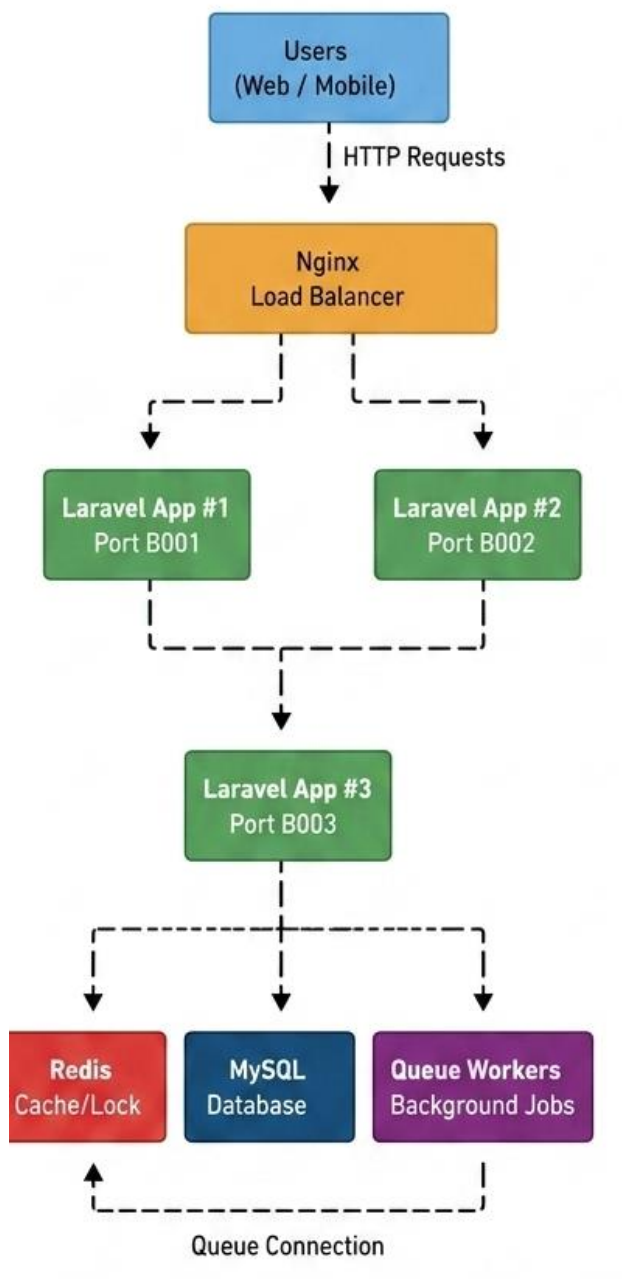
يمثل مخطط حالات الاستخدام (Use Case Diagram) التفاعل بين المستخدم والنظام، حيث يوضح العمليات الأساسية التي يمكن للمستخدم تنفيذها مثل تسجيل الدخول، البحث عن المنتجات، إضافة المنتجات إلى السلة، وإتمام عملية الشراء.

هذا المخطط يساعد في فهم العلاقة بين المستخدم والنظام بشكل بصري ويوضح حدود النظام (System Boundary) بشكل واضح.

Use Case Diagram — High-Performance E-Commerce Backend Engine



Chapter 3 – Architecture Design



يعتمد النظام على معمارية متعددة الطبقات تهدف إلى تحسين الأداء، وزيادة قابلية التوسع و ضمان استقرار النظام تحت الأحمال المرتفعة.

تبدأ دورة حياة الطلب عندما يرسل المستخدم طلباً من خلال واجهة الويب أو تطبيق الهاتف المحمول.

يستقبل خادم nginx الطلبات الواردة ويقوم بتوزيعها على عدة نسخ من تطبيق Laravel باستخدام استراتيجية موازنة الأحمال.

ثم تشغيل ثلاث نسخ مستقلة من التطبيق على المنافذ 8001، 8002، 8003، مما يسمح بتوزيع الطلبات الواردة وتقليل الحمل على كل خادم

تعتمد المنظومة على Redis لتوفير مجموعة من الخدمات الأساسية، وتشمل التخزين المؤقت لبيانات الأكثر استخداماً، وإدارة الجلسات، وتنفيذ الأقفال الموزعة، بالإضافة إلى إدارة طوابير المهام غير المتزامنة.

تتولى قاعدة البيانات MySQL تخزين البيانات الدائمة، مثل بيانات المستخدمين و المنتجات و الطلبات و المخزون، مع استخدام المعاملات Transaction و آليات القفل لضمان اتساق البيانات و سلامتها.

تم نقل المهام طويلة التنفيذ، مثل إرسال الإشعارات وإنشاء التقارير الدورية، إلى Queue Workers تعمل في الخلفية، مما يقلل زمن استجابة الطلبات و يحسن تجربة المستخدم.

ساهم هذا التصميم في تحقيق عدد من المتطلبات غير الوظيفية المهمة، مثل تحسين قابلية التوسع، و تقليل زمن الاستجابة، ومنع تضارب البيانات، وزيادة موثوقية النظام تحت الضغط العالي.

Chapter 4 – Conclusion

حقّق المشروع تطبيقاً كاملاً للمتطلبات غير الوظيفية العشرة بأدلة قابلة للقياس:

- NFR#9: النظام يخدم 100 مستخدم متزامن بـ 0% failure بعد التحسين مقابل 6.64% قبله
- NFR#3: تحسين وقت Login بنسبة 97% بتحويل Jobs إلى Redis Queue
- NFR#6: تحسين Search بنسبة 99% بدمج FullText Index + Cache
- NFR#1: إثبات Race Condition وحلّها بـ Pessimistic Lock + atomic orderBy
- NFR#10: AOP Middleware يُراقب كل endpoint بدون تعديل Business Logic مبدأ Separation of Concerns)

المبدأ المحوري في التصميم: تطبيق كل NFR في المكان المناسب منطقياً — الـ Cache حيث التكرار مرتفع، الـ Lock حيث التضارب محتمل، الـ Transaction حيث الذرية ضرورية، الـ Queue حيث المستخدم لا يحتاج الانتظار.